Single Trajectory GRPO for Cooperative Multi Agent Reinforcement Learning

Brian Ko
University of Washington
kkm97183@uw.edu

Abstract

Proximal Policy Optimization (PPO) is widely used in multi-agent reinforcement learning (MARL), but its reliance on value function learning and GAE-based advantage estimation can limit scalability and introduce value estimation error. Group-Relative Policy Optimization (GRPO) avoids an explicit critic by computing advantages from group-normalized returns, making it attractive for large-scale training. However, GRPO remains underexplored in MARL. We show that a single episode trajectory can be used to compute group advantages by treating different agents as the group for homogeneous cooperative MARL. In this work, we adapt GRPO to cooperative MARL and show that it achieves performance comparable to PPO-style baselines while using up to **8**$\times$ less compute, as shown in Figure 1. Code can be found at `https://github.com/ko120/IGRPO`.

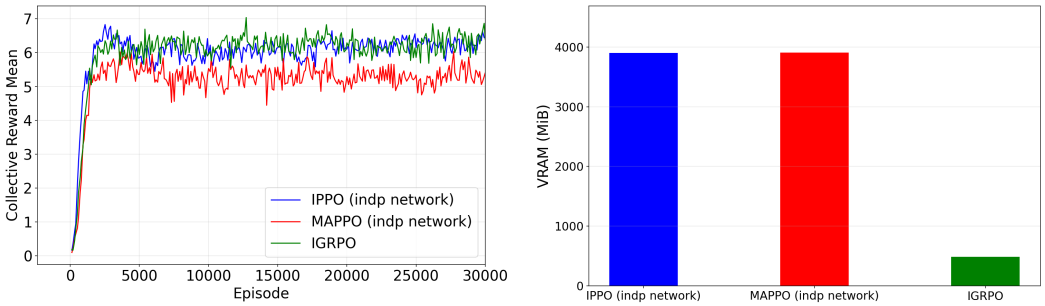

Figure 1: Sample efficiency (left) and VRAM comparison (right) for IPPO, MAPPO, and IGRPO.

1 Introduction

Multi agent reinforcement learning (MARL) is commonly studied under three training execution paradigms: centralized training with decentralized execution (CTDE), decentralized training with decentralized execution (DTDE), and centralized training with centralized execution (CTCE) Jin et al. [2025]. CTCE learns a single centralized controller that aggregates information from all agents to output joint actions, which can be effective but often scales poorly as the number of agents grows Jin et al. [2025]. DTDE instead trains each agent independently using only local information, which improves scalability but frequently suffers from non-stationarity because each agent's learning induces a moving target for the others Jin et al. [2025]. CTDE mitigates this issue by allowing centralized information during training while keeping execution decentralized, and it has become a dominant paradigm in cooperative MARL Jin et al. [2025]. Accordingly, this work focuses on **CTDE**.

Preprint. Under review.

Proximal Policy Optimization (PPO) is widely used in both single agent RL and MARL, but its practical scalability can be limited by critic learning and advantage estimation overhead Schulman et al. [2017]. Group-Relative Policy Optimization (GRPO) provides a scalable alternative by avoiding an explicit critic and computing advantages via group-normalized reward statistics, which has been influential in large-scale post-training Shao et al. [2024]. Despite this, extensions of GRPO to MARL remain relatively underexplored. Recent work demonstrates that GRPO can be adapted to homogeneous cooperative MARL by viewing agents as the "group" and using group statistics to normalize reward, motivating further study beyond LLM settings Zhao and Li [2025].

This paper studies GRPO-style optimization in cooperative MARL and shows that critic-free training can achieve performance comparable to PPO-style baselines while using up to $8\times$ less GPU VRAM.

Our contributions are summarized as follows.

1. We compare common PPO architecture choices in cooperative MARL, including shared encoders for actor–critic networks versus fully separate actor–critic networks.
2. We show that additional information does not always lead to better performance by comparing IPPO vs MAPPO.
3. We demonstrate that critic-free GRPO can match PPO performance while reducing GPU VRAM usage by up to $8\times$.

2 Method

2.1 Preliminaries

We study decentralized partially observable Markov decision processes (DEC-POMDPs) with shared rewards. A DEC-POMDP is defined by $\langle \mathcal{S}, \mathcal{A}, \mathcal{O}, R, P, n, \gamma \rangle$. Here, $\mathcal{S}$ is the state space and $\mathcal{A}$ is the (shared) action space for each agent i. Each agent observes a local observation $o_i = \mathcal{O}(s; i)$ at global state s. The environment dynamics are given by $P(s' \mid s, U)$, the transition probability from s to s' under the joint action $U = (a_1, \ldots, a_n)$ of n agents. The shared reward function is $R(s, U)$, and $\gamma \in (0, 1)$ is the discount factor.

Agents act using a decentralized policy $\pi_\theta(a_i \mid o_i)$ parameterized by θ, producing action a_i from local observation o_i. They jointly optimize the discounted accumulated return

$$J(\theta) = \mathbb{E}_{U_t, s_t}\Big[\sum_{t=0}^{T-1} \gamma^t R(s_t, U_t)\Big], \qquad U_t = (a_t^1, \ldots, a_t^n). \tag{1}$$

2.2 CTDE PPO baselines: IPPO and MAPPO

Under centralized training with decentralized execution (CTDE), agents execute using only local observations, but training may leverage centralized information (e.g., global state, parameter sharing) Jin et al. [2025]. We compare two widely used PPO-style baselines.

IPPO. Independent PPO (IPPO) Yu et al. [2022] trains each agent with only local information and executes using only local information. Although IPPO is often described as decentralized training and decentralized execution (DTDE) Jin et al. [2025], in practice it is common to apply *parameter sharing*, thus making CTDE.

MAPPO. Multi-Agent PPO (MAPPO) Yu et al. [2022] uses a decentralized actor $\pi_\theta(a_{i,t} \mid o_{i,t})$ but trains a centralized critic $V_\phi(s_t)$ that can condition on global information during training, improving stability under non-stationarity Jin et al. [2025].

2.3 GRPO and its MARL adaptation

Group Relative Policy Optimization (GRPO) follows a PPO-like clipped objective, but replaces critic-based advantage estimation with a reward-based, group-normalized advantage. Following the

formulation in Zhao and Li [2025], the GRPO objective is

$$J_{\mathrm{GRPO}}(\theta) = \mathbb{E}_{s,u\sim\pi_{\theta^-}}\Big[\min\Big(r\,\hat{A}_{\pi_\theta}(s,u),\ \mathrm{CLIP}\big(r, 1-\epsilon, 1+\epsilon'\big)\hat{A}_{\pi_\theta}(s,u)\Big)\Big] - \beta\,\mathrm{KL}(\pi_\theta \,\|\, \pi_{\mathrm{ref}}), \tag{2}$$

where $r = \pi_\theta(u \mid s)/\pi_{\theta^-}(u \mid s)$ (importance sampling ratio), θ^- denotes the old policy used to collect samples, and π_{ref} is a reference policy (i.e. initial policy before training) used for KL regularization Zhao and Li [2025].

In PPO, the advantage is defined via value functions,

$$A_\pi(s_t, u_t) = Q_\pi(s_t, u_t) - V_\pi(s_t), \tag{3}$$

typically requiring a critic network to approximate V_π Schulman et al. [2017]. In contrast, GRPO defines a reward-based advantage using group statistics:

$$\hat{A}_\pi(s_t, u_t) = \frac{\sum_{\tau\in K_t}\gamma^\tau R(s_{t+\tau}, u_{t+\tau}) - \mu}{\sigma}, \qquad K_t = \{0, 1, \ldots, T-t\}, \tag{4}$$

where μ and σ are the mean and standard deviation computed across a *group* (in the LLM post training setting, multiple trajectories generated from the same initial state and policy) Shao et al. [2024].

Why global group statistics can fail in MARL. A key assumption in the original LLM post training GRPO formulation is that step rewards have a similar range, which makes the group mean approximately time-invariant and supports the PPO-like baseline interpretation. However, in general MARL, per-step reward scales can vary across agents and over time, violating this assumption. As noted in Zhao and Li [2025], this makes it infeasible to use a single global μ to approximate a value baseline across time in MARL.

Per-time-step cross-agent statistics. When agents are completely homogeneous in cooperative MARL, we can treat the set of agents as the GRPO group and compute group statistics across agents at each time step. Specifically, Zhao and Li [2025] modifies the advantage to

$$\hat{A}'_\pi(s_{i,t}, u_{i,t}) = \frac{\sum_{\tau\in K_t}\gamma^\tau R(s_{i,t+\tau}, u_{i,t+\tau}) - \mu_t}{\sigma_t}, \tag{5}$$

with per-time-step statistics

$$\mu_t = \mathbb{E}_{i\in\mathcal{I}}\Big[\sum_{\tau\in K_t}\gamma^\tau R(s_{i,t+\tau}, u_{i,t+\tau})\Big], \qquad \sigma_t = \sqrt{\mathbb{E}_{i\in\mathcal{I}}\Big[\Big(\sum_{\tau\in K_t}\gamma^\tau R(s_{i,t+\tau}, u_{i,t+\tau}) - \mu_t\Big)^2\Big]}. \tag{6}$$

This per time step normalization provides a more reliable baseline when reward scales are not time invariant.

3 Experiments

3.1 Research Questions

To evaluate our method, we answer the following research questions.

RQ1. How do PPO architecture choices affect performance and efficiency in cooperative MARL?

RQ2. Is more centralized information (e.g., a centralized critic) always beneficial, especially under sparse rewards?

RQ3. Can critic-free GRPO match PPO-style baselines while improving computational efficiency?

3.2 Experimental Setup

Environment. We evaluate our algorithms on a multi-agent gridworld with a fixed 15×15 layout containing static obstacles (e.g., walls or lava). The layout is fixed across episodes, while agents' initial positions are randomized at reset. The objective is for agents to reach a goal square. Each

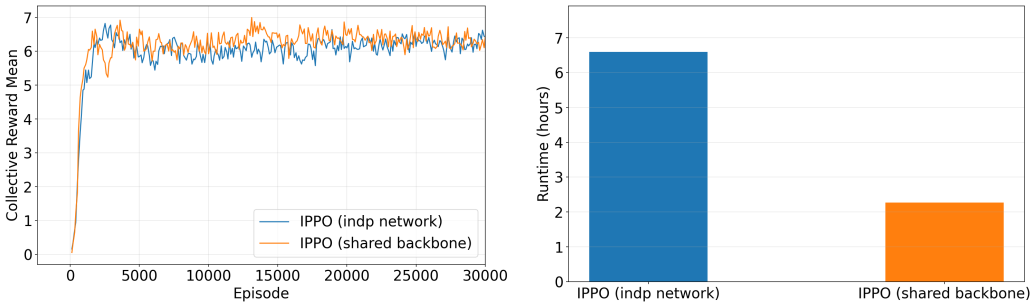

Figure 2: RQ1 results: Sample efficiency (left) and runtime comparison (right) for shared-encoder vs separate-network IPPO variants.

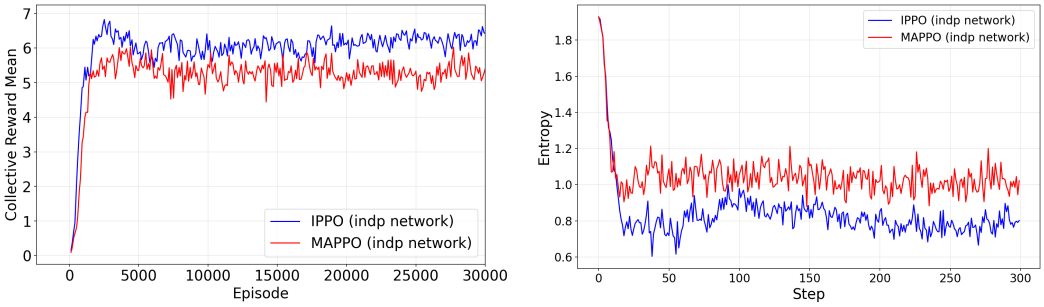

Figure 3: RQ2 results: sample efficiency (left) and policy entropy (right) under sparse rewards for IPPO and MAPPO.

agent receives a local egocentric observation represented as a $(5, 5, 3)$ image patch. The full map is 15×15, and each agent maintains an orientation in {north, south, east, west}. Agents share a discrete action space with seven actions:{turn left, turn right, move forward, pickup, drop, toggle, done (no-op)}. A reward is provided only when the goal is reached:

$$r = 1 \cdot \left(1 - \frac{\texttt{step_count}}{\texttt{max_steps}}\right), \tag{7}$$

and is 0 otherwise, resulting in a sparse-reward learning problem.

Algorithms and variants. In experiment we use IPPO, MAPPO (PPO-style baselines), and GRPO. We adapt parameter sharing for PPO based method for efficiency. We use the same hyperparameters for fair comparison. Implementation details can be found on appendix A.2.

3.3 RQ1: Effect of PPO Architecture Choice

A common design choice in PPO-style MARL is whether the actor and critic share a feature encoder or use separate networks. We compare two IPPO variants: a shared-encoder actor–critic versus fully separate encoder networks.

Result. As shown in Figure 2, the shared-encoder architecture achieves comparable sample efficiency to the separate-network design while using fewer parameters and roughly half the runtime. This indicates that shared representations can support both policy learning and value prediction without degrading performance, while improving memory efficiency.

3.4 RQ2: Is More Information Always Beneficial?

Prior work often reports that MAPPO outperforms IPPO by training a centralized critic that can condition on additional global information Bettini et al. [2024]. We test whether this benefit holds in our sparse reward setting.

Result. Although MAPPO uses a centralized value function, it underperforms IPPO and maintains higher policy entropy during training (Figure 3). This is consistent with the centralized critic being harder to fit under sparse rewards, producing noisier advantage estimates and less stable policy updates. In contrast, IPPO's simpler critic yields more stable learning dynamics and faster entropy reduction.

3.5 RQ3: Can GRPO Match PPO Performance with Less Compute?

A major scalability bottleneck in PPO-style methods is training a critic and computing GAE. If GRPO can match PPO performance without a critic, it provides a more scalable alternative for MARL. We compare IPPO, MAPPO, and GRPO in both learning curves and compute cost.

Result. GRPO matches IPPO level performance while using $8\times$ less compute, measured by reduced GPU VRAM usage (Figure 1). These findings support GRPO as a practical approach to scaling cooperative MARL without relying on critic networks.

4 Limitations

Our study has two main limitations. First, we report results from a single run without multiple random seeds, so performance comparisons may be affected by run-to-run variance. Second, our evaluation is restricted to a single environment, which limits the generality of our conclusions. Future work should include multi-seed experiments and a broader set of cooperative MARL benchmarks to validate robustness and external validity.

References

Matteo Bettini, Amanda Prorok, and Vincent Moens. Benchmarl: Benchmarking multi-agent reinforcement learning. *Journal of Machine Learning Research*, 25(217):1–10, 2024.

Weiqiang Jin, Hongyang Du, Biao Zhao, Xingwu Tian, Bohang Shi, and Guang Yang. A comprehensive survey on multi-agent cooperative decision-making: Scenarios, approaches, challenges and perspectives. *arXiv preprint arXiv:2503.13415*, 2025.

Eric Liang, Richard Liaw, Philipp Moritz, Robert Nishihara, Roy Fox, Ken Goldberg, Sanjay Krishnan, Joseph E. Gonzalez, Michael I. Jordan, and Ion Stoica. Rllib: Abstractions for distributed reinforcement learning. *arXiv preprint arXiv:1712.09381*, 2018.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junjie Song, Xin Bi, Haowei Zhang, Ming Zhang, Yixuan Li, Yuxiao Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of ppo in cooperative multi-agent games. *Advances in neural information processing systems*, 35:24611–24624, 2022.

Zijian Zhao and Sen Li. One step is enough: Multi-agent reinforcement learning based on one-step policy optimization for order dispatch on ride-sharing platforms. *arXiv preprint arXiv:2507.15351*, 2025.

Table 1: Training hyperparameters for GRPO experiments.

Parameter	Value
Discount factor γ	0.99
Learning rate	0.001
Train batch size	4000
Minibatch size	250
Epochs per iteration	4
PPO clip parameter ϵ	0.2
Value loss coefficient	0.0
Entropy coefficient	0.01
Gradient clipping	0.5
Policy sharing (CTDE)	enabled
Backbone sharing (`share_backbone`)	False
Total episodes	30000

Table 2: Training hyperparameters for PPO experiments.

Parameter	Value
Discount factor γ	0.99
Learning rate	0.001
Train batch size	4000
Minibatch size	250
Epochs per iteration	4
GAE λ	0.95
PPO clip parameter ϵ	0.2
Value loss coefficient	0.5
Entropy coefficient	0.01
Gradient clipping	0.5
Policy sharing (CTDE)	enabled
Backbone sharing (`share_backbone`)	False
Total episodes	30000

A Implementation Details

A.1 Code

We implement our code base on RLlib (Ray) Liang et al. [2018].

A.2 Hyperparameters and Training Configuration

Table 1 lists the GRPO hyperparameters. We use parameter sharing (CTDE) so all agents share one policy. In GRPO mode, we set the value loss coefficient to zero since no critic is trained.

Table 2 summarizes the PPO configuration;